

THOUGHTBOOK

Detailed Project Workflow, Architecture & Technology Stack

Project Type	Full-stack web application
Frontend	React + Vite + Tailwind CSS
Backend	FastAPI (Python)
Database	MySQL + PyMySQL
Authentication	JWT + Google OAuth 2.0 / OpenID Connect
Document Purpose	Reference for development, viva, portfolio and future expansion

This document consolidates the current ThoughtBook architecture and the authentication workflow discussed during development. It is intentionally structured so that each layer can be explained independently during a technical interview or project viva.

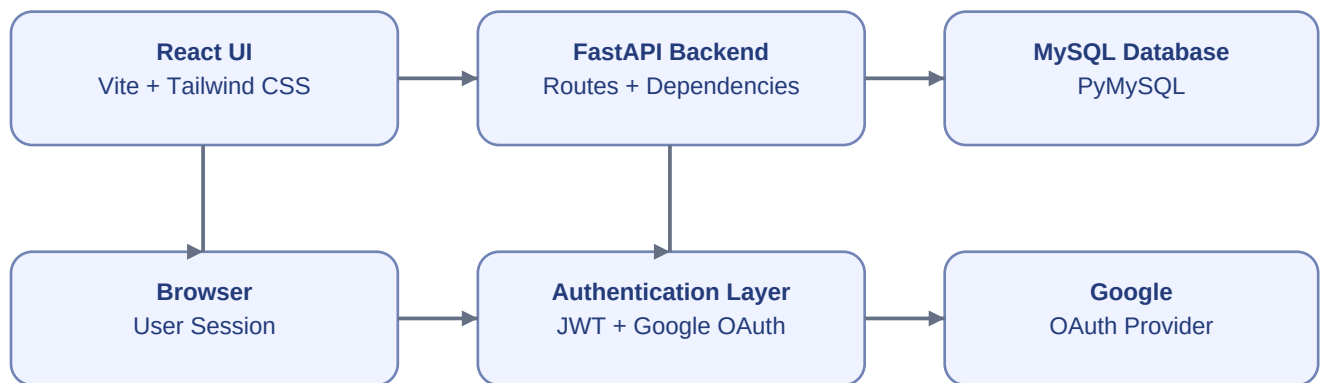
1. Project Overview

ThoughtBook is a full-stack application designed around user authentication and a personal thought/book-style experience. The application separates the user interface, API/business logic, authentication, and persistence layers.

Primary goals

- Provide a responsive React-based user interface.
- Expose clean REST-style API endpoints through FastAPI.
- Support traditional email/password authentication.
- Support Sign in with Google using OAuth/OpenID Connect.
- Persist user information in MySQL.
- Use JWTs to maintain authenticated API sessions after login.
- Keep frontend and backend responsibilities clearly separated.

High-level architecture



Core request path: Browser → React → FastAPI → authentication/business logic → MySQL. For Google login, FastAPI redirects the browser to Google and later receives the OAuth callback.

2. Technology Stack

Layer	Technology	Role
Frontend	React	Component-based UI and application state/interaction
Build tool	Vite	Fast development server and production bundling
Styling	Tailwind CSS	Responsive utility-first styling and layout
Backend	FastAPI	REST API, routing, validation, dependencies and auth endpoints
Language	Python	Backend implementation
Validation	Pydantic	Request/response schema validation
Authentication	JWT	Stateless access-token based authentication
OAuth	Authlib	Google OAuth/OpenID Connect integration
Database	MySQL	Persistent user/account data
DB driver	PyMySQL	Python-to-MySQL connectivity
Configuration	Environment variables / .env	Secrets and deployment-specific configuration
Frontend routing	React Router	Client-side navigation between pages
Email/login identity	Google OpenID Connect	Google identity information for Sign in with Google

Why this stack works

- **React + Vite:** keeps the frontend component-driven and fast during development.
- **Tailwind:** makes responsive UI iteration quick without maintaining a large custom stylesheet.
- **FastAPI:** provides typed request handling, dependency injection and automatic API documentation.
- **MySQL:** provides durable relational storage for users and future application entities.
- **JWT:** is appropriate for authenticating subsequent API requests after credentials are verified.
- **Authlib + Google:** handles the OAuth authorization-code flow rather than manually implementing the protocol.

3. Frontend Workflow

The frontend is responsible for presentation, user interaction, navigation and sending HTTP requests to the FastAPI server.

Typical frontend structure

```
src/ ■■■ components/ ■■■ pages/ ■ ■■■ Login.jsx ■ ■■■ ... ■■■ App.jsx ■■■ main.jsx ■■■ ...
```

Login page responsibilities

- Render email/password inputs and the login action.
- Render the **Login with Google** button.
- For normal login, send credentials to the backend API.
- For Google login, navigate the browser to the backend's Google-login endpoint.
- After successful authentication, store/use the returned authentication state according to the chosen frontend session strategy.

Google button connection

The Google button should not call Google's token endpoints directly from the React component. Instead, the browser is sent to the FastAPI OAuth-start route. FastAPI constructs the Google authorization URL and redirects the browser.

```
const handleGoogleLogin = () => { window.location.href =  
  "http://localhost:8000/thoughtbook/login/google"; };
```

When frontend and backend are deployed separately, replace the local backend origin with the deployed API origin. The Google OAuth redirect URI configured in Google Cloud must exactly match the callback URL used by FastAPI.

4. FastAPI Backend Workflow

The backend is the security and business-logic boundary. It receives requests, validates data, performs authentication, talks to MySQL, and returns responses.

Suggested logical modules

```
backend/ ■■■ main.py ■■■ routes/ ■ ■■■ login.py ■ ■■■ ... ■■■ auth/ ■ ■■■ ... ■■■ database/  
■ ■■■ ... ■■■ server/config files
```

FastAPI concepts already used

- Pydantic validation and **response_model**.
- Custom exceptions and HTTP error responses.
- Middleware for cross-cutting request processing.
- Dependency injection for reusable authentication/database logic.
- HTTPBearer for receiving Bearer tokens.
- JWT generation and decoding.
- OAuth integration through Authlib.

5. Email/Password Login Workflow

1	User enters email and password	React Login page collects credentials.
2	Frontend sends login request	HTTP request is sent to FastAPI.

3	Backend validates request	Pydantic/schema validation checks input shape.
4	Database lookup	Backend queries MySQL for the matching user.
5	Password verification	Stored password hash is compared with the supplied password using the project's password-verification function.
6	JWT creation	If valid, FastAPI generates a signed JWT with an expiration time.
7	Response to frontend	Frontend receives authentication result/token according to the application's session design.
8	Protected requests	Frontend sends Authorization: Bearer to protected endpoints.
9	Token verification	FastAPI HTTPBearer/dependency decodes and verifies the JWT before allowing access.

JWT concept

A JWT contains claims such as user identity and expiration and is cryptographically signed. The server should verify the signature and expiration before trusting the claims. The secret/signing key must never be hard-coded into source control.

6. Google Login / OAuth Workflow

ThoughtBook uses an OAuth authorization flow with Google. Authlib is used on the FastAPI side to manage the OAuth client interaction.

1. User clicks Google login	React sends the browser to the backend Google-login route.
2. FastAPI creates authorization request	The backend uses the registered Google OAuth client and requested scopes.
3. Redirect to Google	Browser is redirected to Google's authorization/consent page.
4. User authenticates	Google authenticates the user and asks for the requested permissions when required.
5. Google redirects back	Google redirects the browser to the exact callback URI configured for the application.
6. FastAPI callback	The callback route exchanges the authorization code for Google tokens/user information.
7. Identify/create local user	Backend searches MySQL using the Google identity/email and creates a local account if the application requires it.
8. Create ThoughtBook JWT	After local authentication succeeds, backend generates its own JWT.
9. Return to frontend	Backend redirects/returns the application to the frontend with the application's authentication state handled securely.
10. Authenticated application	Frontend can now access protected FastAPI endpoints using the ThoughtBook authentication mechanism.

OAuth configuration used in the project

- **Provider:** Google
- **Client library:** Authlib Starlette integration.
- **Scopes:** openid, email, profile.
- **Configuration:** GOOGLE_CLIENT_ID and GOOGLE_CLIENT_SECRET are supplied through environment configuration.
- **Callback generation:** FastAPI can use `request.url_for(...)` to construct the registered callback route.
- **Session/state:** OAuth state is important for CSRF protection; mismatched-state errors indicate the browser's OAuth state did not match the server-side expectation.

7. Google OAuth Errors Encountered & What They Mean

Error	Meaning / Fix
itsdangerous missing	Install the required itsdangerous dependency used by Starlette session/state handling.
MismatchingStateError / CSRF Warning	OAuth state stored for the login attempt did not match the callback. Check session middleware, consistent host/origin, cookies, and that the flow is not being interrupted.
Redirect URI mismatch	The callback URI sent by the application does not exactly match a URI authorized in Google Cloud. Scheme, host, port and path all matter.
404 on OAuth routes	FastAPI route/router was not registered correctly, the path prefix was wrong, or the requested URL did not match the declared route.

Error	Meaning / Fix
KeyError: 1 in google_callback	The database result was dictionary-like rather than a tuple/list, so user[1] attempted to access key 1. Access the returned column by its actual dictionary key, or configure the cursor/result shape consistently.

8. MySQL Database Design

The project uses MySQL for persistent user data. The database discussed during development is named **thoughtbook_login**.

User table

Column	Purpose	Important property
id	Unique user identifier	Auto-increment primary key
email	User login/identity	Unique
password / credential field	Local authentication credential	Store a secure password hash, not plaintext
Other profile fields	Application-specific identity/profile	Extend as needed

Important database principle

Keep database access isolated in functions/modules such as `create_user()`, `get_user_by_id()`, and `get_user_by_email()`. This makes route code easier to test and prevents SQL logic from being scattered across the application.

Tuple vs dictionary result issue

PyMySQL cursors can be configured to return rows in different shapes. If a query returns a tuple, `user[1]` may refer to the second selected column. If it returns a dictionary, `user[1]` is interpreted as a dictionary key and can raise `KeyError`. The route should match the cursor configuration and use explicit column names where possible.

9. API Request/Response Workflow

```
Browser ↓ React component / service ↓ HTTP request ↓ FastAPI route ↓ Pydantic validation ↓  
Dependency / authentication ↓ Business logic ↓ Database function ↓ MySQL ↓ Response model / JSON ↓  
React UI update
```

Protected endpoint pattern

- Client sends **Authorization: Bearer <JWT>**.
- HTTPBearer extracts the credentials.
- A dependency verifies/decode the token.
- The endpoint receives the authenticated user identity.
- Business logic executes only after authorization succeeds.

10. Security Checklist

- Never commit `GOOGLE_CLIENT_SECRET`, JWT signing secrets, database passwords or API keys to GitHub.
- Use environment variables for secrets in local and production environments.
- Store password hashes rather than plaintext passwords.
- Set a reasonable JWT expiration time; the current project work used approximately 30 minutes for token expiry.
- Validate JWT signature and expiration on every protected request.
- Use HTTPS in production.
- Configure CORS narrowly to allow only the intended frontend origins.
- Use secure cookie settings if cookies are used for OAuth/session state.
- Keep Google redirect URIs exact and environment-specific.
- Do not trust user-provided identity fields merely because they appear in a request.
- Use parameterized SQL queries to prevent SQL injection.

Production note

For a production-grade implementation, consider short-lived access tokens plus refresh-token rotation, or an appropriately secured HTTP-only cookie session architecture. The exact choice depends on the deployment and threat model.

11. Development → Production Workflow

Local development

- Start the FastAPI development server.
- Start the Vite development server.
- Run MySQL locally and ensure the ThoughtBook database/table exists.
- Use local environment variables for database and Google OAuth credentials.

Production architecture

- Frontend is built into optimized static assets and hosted on a frontend hosting platform.
- FastAPI runs as a backend service/API server.
- MySQL runs as a managed or separately hosted database.
- Google OAuth callback is changed to the production backend callback URL.
- Frontend calls the production API origin rather than localhost.

Environment variables

```
GOOGLE_CLIENT_ID=... GOOGLE_CLIENT_SECRET=... JWT_SECRET=... DATABASE_HOST=... DATABASE_USER=...  
DATABASE_PASSWORD=... DATABASE_NAME=thoughtbook_login FRONTEND_URL=...
```

12. Complete End-to-End User Workflow

A. New user using Google

- User opens ThoughtBook.
- React renders the login page.
- User clicks Login with Google.
- Browser navigates to FastAPI's Google-login route.
- FastAPI redirects to Google.
- User authenticates with Google.
- Google redirects to the registered FastAPI callback.
- FastAPI exchanges the authorization code and reads the user's Google identity.
- FastAPI finds or creates the local MySQL user.
- FastAPI issues a ThoughtBook JWT.
- Frontend becomes authenticated and can call protected APIs.

B. Existing local user

- User enters email/password.
- React sends credentials to FastAPI.
- FastAPI validates input and queries MySQL.
- Password hash is verified.
- FastAPI issues a JWT.
- React uses the authentication state for subsequent protected requests.

C. Protected API request

- React sends the JWT in the Authorization header.
- FastAPI's HTTPBearer/dependency layer extracts and verifies it.
- The endpoint obtains the authenticated user identity.
- Business logic reads/writes the database.
- JSON response is returned to React.

13. Responsibility Map

Component	Should be responsible for
React Login.jsx	Rendering login UI and initiating login actions; avoid embedding database/auth secrets.
React Router	Client-side page navigation.
FastAPI routes	HTTP endpoint definitions, request parsing and response handling.
Auth module	Password verification, JWT creation/verification and OAuth-related logic.
Database module	Connections, SQL queries and mapping database rows to application data.
Pydantic models	Input/output data contracts and validation.

Component	Should be responsible for
MySQL	Durable relational storage.
Google OAuth	External identity authentication and authorization.

14. Recommended Next Development Steps

- **Connect the Google button:** set `window.location.href` to the backend OAuth-start route.
- **Finish callback handling:** resolve the dictionary/tuple database result issue and return/redirect with the final auth state.
- **Centralize API calls:** create a small frontend API service instead of hard-coding URLs in multiple components.
- **Protect routes:** add a React protected-route pattern for authenticated pages.
- **Improve session security:** choose a consistent token/cookie strategy and document it.
- **Database layer:** introduce SQLAlchemy/Alembic later if schema complexity grows; it is not mandatory for the current basic project.
- **Testing:** add pytest tests for login, JWT verification, OAuth callback logic and database helpers.
- **Deployment:** configure production CORS, HTTPS, environment variables and Google OAuth production redirect URIs.

Suggested learning order

React routing → API integration → JWT authentication → OAuth flow → database abstraction → testing → deployment/security hardening.

15. One-Minute Viva Explanation

ThoughtBook is a full-stack application built with React on the frontend and FastAPI on the backend. React provides the responsive UI using Vite and Tailwind CSS, while FastAPI exposes the application's APIs and handles validation, authentication and business logic. MySQL stores user information and PyMySQL connects the Python backend to the database. The project supports both normal login and Google login. For Google login, the browser is redirected from React to FastAPI, FastAPI performs the OAuth authorization flow with Google, receives the callback, identifies or creates the local user in MySQL, and then creates a ThoughtBook JWT. Subsequent protected requests use the JWT through a Bearer Authorization header, which FastAPI verifies before granting access.

Architecture in one line

React/Vite/Tailwind → FastAPI/Pydantic/Auth → JWT or Google OAuth → MySQL/PyMySQL → JSON response → React UI

End of document